

JavaScript DOM Manipulation Lecture Notes

1. What is the DOM?

Meaning: DOM stands for Document Object Model. When a web browser loads an HTML document, it creates a hierarchical, tree-like structure of all the elements on the page. This tree is the DOM. It acts as an interface between JavaScript and the HTML structure.

Real-Life Example: Think of the DOM like a family tree or an organizational chart. The `<html>` tag is the CEO (the root), and `<head>` and `<body>` are the managers. Inside `<body>`, you have paragraphs `<p>` and headings `<h1>` as the employees.

The Document Object

Meaning: The `document` object is the root of the DOM tree. It represents the entire HTML page. If you want to access or change anything on the webpage using JavaScript, you must start by calling the `document` object.

What is DOM Manipulation?

Meaning: DOM manipulation is the process of using JavaScript to dynamically change the HTML structure, styling, or content of a webpage after it has already loaded. This is how websites become interactive (e.g., clicking a button to show a hidden menu).

2. Methods to Access Elements

Before you can change an HTML element, you must first "select" or "find" it using JavaScript.

`getElementById()`

Meaning: Selects a single element based on its unique `id` attribute. This is the fastest and most common way to grab a specific element.

```
Code:
const myTitle = document.getElementById('main-title');
console.log(myTitle);
```

`getElementsByName()`

Meaning: Selects all elements that share a specific class name. It returns an `HTMLCollection` (which looks like an array).

```
Code:
const buttons = document.getElementsByClassName('btn-
primary');
```

```
console.log(buttons[0]); // Accesses the first button with
this class
```

querySelector()

Meaning: Selects the FIRST element that matches a specific CSS selector (like ``.class``, ``#id``, or ``tag``). It is highly versatile.

Code:

```
const firstParagraph = document.querySelector('p');
const specialBox = document.querySelector('.special-box');
```

querySelectorAll()

Meaning: Selects ALL elements that match a CSS selector. It returns a `NodeList` (which you can loop through using `forEach`).

Code:

```
const allItems = document.querySelectorAll('li.menu-item');
allItems.forEach(item => console.log(item));
```

3. Modifying Content and Styles

Changing Content: `innerText` vs. `innerHTML`

- `innerText`: Reads or changes only the visible text inside an element. It ignores HTML tags.
- `innerHTML`: Reads or changes the text AND the HTML tags inside an element. Be careful using this with user input due to security risks.

Code:

```
const myDiv = document.getElementById('content-box');
```

```
// Changes just the text
myDiv.innerText = 'Hello, World!';
```

```
// Adds bold HTML formatting
myDiv.innerHTML = '<strong>Hello, World!</strong>';
```

Changing CSS Styles directly

Meaning: You can modify inline CSS styles directly via the ``.style`` property. CSS properties with hyphens (like ``background-color``) must be written in camelCase (like ``backgroundColor``).

Code:

```
const header = document.querySelector('h1');
header.style.color = 'blue';
```

```
header.style.backgroundColor = 'lightgray';
header.style.fontSize = '24px';
```

4. Managing CSS Classes with `classList`

Instead of changing inline styles one by one, it is better practice to add or remove predefined CSS classes using the `classList` property.

- `classList.add()`: Adds a class to the element.
- `classList.remove()`: Removes a class from the element.
- `classList.toggle()`: If the class exists, it removes it. If it doesn't exist, it adds it. (Perfect for Dark Mode buttons!)

```
Code:
const box = document.getElementById('my-box');

box.classList.add('highlight'); // Adds the highlight class
box.classList.remove('hidden'); // Removes the hidden class
box.classList.toggle('active'); // Toggles the active class
on/off
```

5. Adding New Elements to the DOM

You can create entirely new HTML elements from scratch using JavaScript and place them onto the webpage.

- `document.createElement('tag')`: Creates the HTML element in memory.
- `parent.appendChild(child)`: Attaches the newly created element to an existing parent element on the screen.

Example: Creating a Single List Item

```
Code:
// 1. Select the parent ul
const ul = document.getElementById('my-list');

// 2. Create a new li element
const newLi = document.createElement('li');

// 3. Add text to the li
newLi.innerText = 'I am a new list item!';

// 4. Attach the li to the ul
ul.appendChild(newLi);
```

Example: Appending a Whole List using a For Loop

```
Code:
const ul = document.getElementById('my-list');
const fruits = ['Apple', 'Banana', 'Mango', 'Orange'];

for (let i = 0; i < fruits.length; i++) {
  // Create a new li for each fruit in the array
  let li = document.createElement('li');
  li.innerText = fruits[i];

  // Append it to the ul
  ul.appendChild(li);
}
```

6. DOM Traversal (Parent, Child, Sibling)

Meaning: DOM Traversal means navigating through the DOM tree to find related elements. If you have an element, you can easily find its parent, its children, or the elements sitting next to it (siblings).

- `.parentElement`: Targets the immediate parent of the current element.
- `.children`: Targets all child elements inside the current element (returns an `HTMLCollection`).
- `.previousElementSibling`: Targets the sibling immediately before the current element.
- `.nextElementSibling`: Targets the sibling immediately after the current element.

```
Code:
const currentItem = document.getElementById('item-2');

// Finding the parent
const parentList = currentItem.parentElement;
console.log(parentList); // Logs the <ul> element

// Finding children
const allChildren = parentList.children;
console.log(allChildren); // Logs all <li> elements inside the
                           <ul>

// Finding siblings
const previousItem = currentItem.previousElementSibling; //
                                                           item-1
const nextItem = currentItem.nextElementSibling; // item-3
```

7. Event Listeners

Meaning: An Event Listener is a procedure in JavaScript that waits for an event to occur (like a user clicking a mouse or pressing a key) and then executes a function in response.

HTML inline onClick vs. addEventListener

- `onClick`: The older way of adding events directly in HTML (e.g., `<button onclick="myFunction()">Click Me</button>`) or directly to the DOM object (`btn.onclick = function() {}`). It is generally discouraged because it mixes HTML and JS, and you can only assign ONE `onClick` event per element.
- `addEventListener()`: The modern, preferred method. It keeps JS separate from HTML and allows you to attach MULTIPLE event listeners to a single element without overwriting them.

Code:

```
const btn = document.getElementById('my-btn');

// Bad Practice (onClick overwrites previous onClicks)
btn.onclick = () => console.log('This will be overwritten');
btn.onclick = () => console.log('Only this runs');

// Good Practice (addEventListener)
btn.addEventListener('click', () => {
  console.log('Button was clicked!');
});
```

Common Event Types

- `'click'`: Fires when the element is clicked.
- `'mouseover'`: Fires when the mouse pointer moves over an element.
- `'mouseout'`: Fires when the mouse pointer moves off an element.
- `'keydown'`: Fires when a keyboard key is pressed down.
- `'keyup'`: Fires when a keyboard key is released.
- `'submit'`: Fires when a form is submitted.

Example: Keyboard Events

Code:

```
document.addEventListener('keydown', (event) => {
  if (event.key === 'Enter') {
    console.log('The Enter key was pressed!');
  } else {
    console.log(`You pressed: ${event.key}`);
  }
});
```

```
    }  
  });  
};
```

8. Form Events

Meaning: Forms are a crucial part of web development. When a user submits a form (e.g., clicking the submit button or pressing Enter), the browser tries to refresh the page by default. We use the 'submit' event listener and `event.preventDefault()` to stop the page from refreshing so we can handle the data with JavaScript.

Example: Simulating a Google Search Form

Imagine an HTML form with an input field and a search button:

```
<form id="search-form">  
  <input type="text" id="search-input" placeholder="Search Google..." />  
  <button type="submit">Search</button>  
</form>  
<h2 id="results"></h2>
```

Code:

```
const form = document.getElementById('search-form');  
const input = document.getElementById('search-input');  
const resultsHeader = document.getElementById('results');  
  
form.addEventListener('submit', (event) => {  
  // 1. Stop the page from refreshing  
  event.preventDefault();  
  
  // 2. Get the value the user typed into the input  
  const searchTerm = input.value;  
  
  // 3. Update the DOM with the result  
  if (searchTerm === '') {  
    resultsHeader.innerHTML = "Please enter a search  
term!";  
  } else {  
    resultsHeader.innerHTML = `Showing results for:  
${searchTerm}`;  
  }  
  
  // 4. Clear the input field after searching  
  input.value = '';  
});
```